

AJAX

XML

eXtensible Markup Language — расширяемый язык разметки;

```
<font color="#069"><person></font>  
  <font color="#069"><firstname></font>Subbu<font  
color="#069"></firstname></font>  
  <font color="#069"><lastname></font>Allamaraju<font  
color="#069"></lastname></font>  
  <font color="#069"></person></font>
```

JSON

JavaScript Object Notation

```
{  
  <font color="#069">"firstName"</font> : <font color="#069">"Subbu"</font> ,  
  <font color="#069">"lastName"</font> : <font color="#069">"Allamaraju"</font>  
};
```

Методы работы с JSON

в PHP5 есть встроенные функции `json_encode()` и `json_decode()`

`Json` преобразовывает в объект или массив

`JSON.parse` — читает объекты из строки в формате JSON.

`JSON.stringify` — превращает объекты в строку в формате JSON, используется, когда нужно из JavaScript передать данные по сети.

Пример преобразования строки JSON в объект

```
var jsontext =  
'{"firstname":"Jesper","surname":"Aaberg","phone":["555-0100","555-0120"]}';  
  
var contact = JSON.parse(jsontext);  
  
alert(contact.surname + ", " + contact.firstname);  
  
alert(typeof(contact));
```

Пример

преобразование массива в строку JSON с помощью JSON.stringify, а затем преобразование этой строки обратно в массив с помощью JSON.parse.

```
var arr = ["a", "b", "c"];
var str = JSON.stringify(arr);
alert(str);
alert("<br/>");

var newArr = JSON.parse(str);

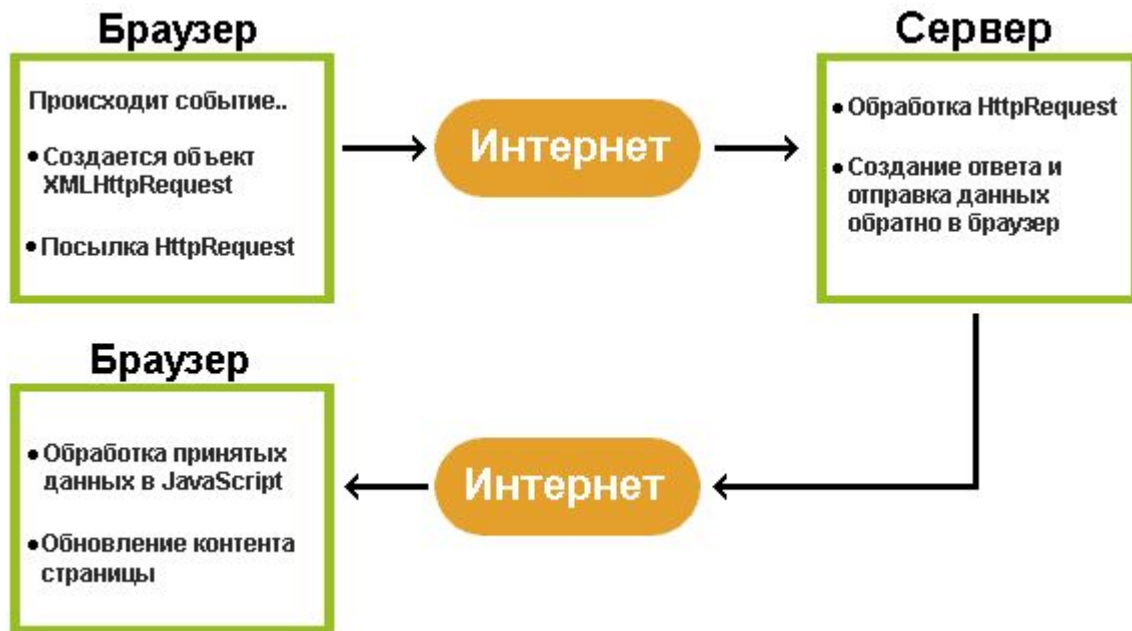
while (newArr.length > 0) {
    alert(newArr.pop() + "<br/>");
}
```

Существует множество сервисов для валидации и преобразования в JSON и XML форматы

<http://www.freeformatter.com/json-validator.html>

AJAX

AJAX = Asynchronous JavaScript and XML



Самое основное - это XMLHttpRequest - объект, для обмена информацией с сервером без перезагрузки страницы.

var xhttp = new XMLHttpRequest(); - создание

```
var xhttp; // код для старых браузеров
```

```
if (window.XMLHttpRequest) {
```

```
    xhttp = new XMLHttpRequest();
```

```
} else {
```

```
// code for IE6, IE5
```

```
xhttp = new ActiveXObject("Microsoft.XMLHTTP"); }
```

Отправка запроса на сервер методы open() и send()

```
xhttp.open("GET", "ajax_info.txt", true);
```

```
xhttp.send();
```

open(method, url, async) - method: тип запроса: GET or POST, url: файл на сервере, async: true (asynchronous) or false (synchronous)

send() - отправка запроса на сервер(для GET)

send(string) - для POST

Get или POST?

//GET быстрее и проще, чем POST.

POST для отправки большого объема информации.

POST более защищен и надежен.

```
xhttp.open("GET", "demo_get.asp", true); //GET запрос
```

```
xhttp.send();
```

```
xhttp.open("POST", "demo_post.asp", true); //POST запрос
```

```
xhttp.send();
```

```
<h2>AJAX</h2>
```

```
<button type="button" onclick="loadDoc()">Request data</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function loadDoc() {
```

```
    var xhttp = new XMLHttpRequest();
```

```
    xhttp.onreadystatechange = function() {
```

```
        if (xhttp.readyState == 4 && xhttp.status == 200) {
```

```
            document.getElementById("demo").innerHTML = xhttp.responseText; }    }
```

```
    xhttp.open("GET", "demo_get2.asp?fname=Henry&lname=Ford", true);
```

```
    xhttp.send(); }
```

```
</script> - пример
```

Чтобы отправлять данные методом POST - наподобие отправки данных с HTML формы, добавьте заголовок HTTP с помощью метода `setRequestHeader()`. Укажите данные, которые Вы хотите отправить, посредством метода `send()`:

```
xmlhttp.open("POST","ajax_test.php",true);
```

```
xmlhttp.setRequestHeader("Content-type","application/x-www-form-urlencoded");
```

```
xmlhttp.send("fname=Иван&lname=Петров");
```

`setRequestHeader(заголовок,значение)` Добавляет HTTP заголовки к запросу.

заголовок: указывает имя заголовка

значение: указывает значение заголовка

Ассинхронный запрос - не нужно ожидать ответа, можно исполнять другие скрипты или действия.

`xhttp.open("GET", "ajax_test.asp", true);` - ассинхронный запрос

`setRequestHeader(name, value)` - HTTP заголовок

`xhr.setRequestHeader('Content-Type', 'application/json');` - пример заголовка

Ответ от сервера

responseText - ответ в виде строки

```
document.getElementById("demo").innerHTML = xhttp.responseText;
```

responseXML - ответ в виде XML

Когда серверу посылается запрос, мы хотим выполнить некоторые действия на основе ответа.

Событие `onreadystatechange` происходит каждый раз, когда свойство `readyState` (состояние готовности) изменяется.

Свойство `readyState` содержит состояние запроса `XMLHttpRequest`.

Три важных свойства объекта `XMLHttpRequest`:

`onreadystatechange` Хранит функцию (или имя функции), которая вызывается автоматически каждый раз, когда изменяется свойство `readyState`

`readyState` Содержит состояние объекта `XMLHttpRequest`. Изменяется от 0 до 4:

0: запрос не инициализирован

1: установлено соединение с сервером

2: запрос получен

3: обработка запроса

4: запрос завершен и ответ готов

`status` 200: "ОК (все хорошо)"

404: Страница не найдена

В событии `onreadystatechange` мы указываем, что произойдет, когда ответ с сервера будет получен и готов для обработки.

Когда состояние `readyState` равно 4 и статус (`status`) содержит значение 200, это означает, что ответ готов:

```
function loadDoc() {  
  
    var xhttp = new XMLHttpRequest();  
  
    xhttp.onreadystatechange = function() {  
  
        if (xhttp.readyState == 4 && xhttp.status == 200) {  
  
            document.getElementById("demo").innerHTML = xhttp.responseText;  
  
        }  
  
    }  
  
}
```

Событие `onreadystatechange` происходит четыре раза - по одному на каждое изменение состояния запроса `readyState`.

Функция обратного вызова(callback function)

Функция обратного вызова указывается в качестве параметра для другой функции.

Если Вы хотите выполнить более одной задачи AJAX на веб сайте, следует создать ОДНУ стандартную функцию для создания объекта XMLHttpRequest и вызывать ее для каждого AJAX задания.

Эта функция должна содержать адрес URL и действия, которые необходимо выполнить при возникновении события onreadystatechange (которые возможно будут различными при каждом вызове):

```
function myFunction()  
  
{  loadXMLDoc("ajax_info.txt",function()  
  
    {  if (xmlhttp.readyState==4 && xmlhttp.status==200)  
  
        {  document.getElementById("myDiv").innerHTML=xmlhttp.responseText;  }  
  
    });  
  
}
```

Пример

http://www.w3schools.com/ajax/ajax_php.asp

Функция `showHint()`

Когда пользователь печатает очередной символ в поле ввода выше, выполняется функция `"showHint()"`. Функция вызывается благодаря возникновению события `"onkeyup"`:

<script>

```
function showHint(str) {  
  
    if (str.length == 0) {  
  
        document.getElementById("txtHint").innerHTML = "";  
  
        return;  
  
    } else {  
  
        var xmlhttp = new XMLHttpRequest();  
  
        xmlhttp.onreadystatechange = function() {  
  
            if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {  
  
                document.getElementById("txtHint").innerHTML = xmlhttp.responseText;  
  
            }  
  
        }  
  
    }  
}
```

```
}
```

```
    xmlhttp.open("GET", "gethint.php?q=" + str, true);
```

```
    xmlhttp.send();
```

```
}
```

```
}
```

```
</script>
```

```
<body>
```

```
<p><b>Start typing a name in the input field below:</b></p>
```

```
<form> First name: <input type="text" onkeyup="showHint(this.value)"> </form>
```

```
<p>Suggestions: <span id="txtHint"></span></p>
```

```
</body>
```

Пояснение исходного кода:

Если поле ввода является пустым (`str.length==0`), функция очищает содержимое контейнера `txtHint` (места вставки предложений) и завершает выполнение.

Если поле ввода не является пустым, функция `showHint()` осуществляет следующие действия:

- Создает объект `XMLHttpRequest`
- Создает функцию, которая будет выполнена, когда будет получен ответ с сервера
- Посылает запрос на сервер, в качестве адресата запроса выступает файл с серверным `php`-скриптом
- Обратите внимание, что параметр (`q`) добавляется к адресу URL (и содержит текст, введенный в поле ввода)

Страница на сервере, к которой обращается функция JavaScript выше, является PHP файлом с именем "gethint.php".

на сайте w3s

AJAX на JQuery

`$("селектор").load(url, данные, функция)`

селектор выбирает элемент, в котором будет отображен результат запроса.

url адрес файла, который будет запрошен у сервера с помощью AJAX.

данные которые будут переданы запрошенному файлу. Если Вы хотите запросить файл, при этом не передавать никаких данных, оставьте данный параметр пустым.

функция имя функции, которая будет вызвана после выполнения запроса.


```
$("#but1").click(function(){  
    $("#par1").load("testfile.txt");  
})
```

В примере мы пересылаем скрипту add.php переменную x со значением 4 и переменную y со значением 5 скрипт в свою очередь принимает эти переменные, производит их сложение и отправляет результат обратно.

```
$(document).ready(function(){  
  
    $("#but1").click(function(){  
        $("#par1").load("add.php","x=4&y=5");  
    })  
  
});
```

Сопровождающие функции

Сопровождающие функции позволяют выполнять различный код в зависимости от состояния выполнения AJAX запроса.

Метод **ajaxSend()** выполняет переданный в него код во время отправления AJAX запроса.

Метод **ajaxComplete** выполняет переданный в него код после завершения выполнения AJAX запроса (*при этом неважно произошла ошибка или нет*).

```
$(document).ready(function(){  
  
    $("#par1").ajaxSend(function(){  
        $("#img1").css("display","block");  
    })  
    $("#par1").ajaxComplete(function(){  
        $("#img1").css("display","none");  
    })  
    $("#but1").click(function(){  
        $("#par1").load("add.php","x=10&y=100");  
    })  
  
});
```

Методы

Метод `ajaxSuccess` выполняет переданный в него код, если выполнение AJAX запроса завершается успешно.

Метод `ajaxError` выполняет переданный в него код, если выполнение AJAX запроса завершается с ошибкой.

Методы `ajaxStart` и `ajaxStop` используются для сопровождения группы AJAX запросов.

Метод `ajaxStart` выполняет переданный в него код при отправлении первого запроса из группы.

Метод `ajaxStop` выполняет переданный в него код при завершении последнего запроса из группы.

Низкоуровневые AJAX запросы

Низкоуровневые AJAX запросы предоставляют более широкую функциональность, но более сложны в использовании.

```
$("#but1").click(function(){  
    $.ajax({  
        url:"add.php",  
        data:"x=4&y=5",  
        success:function(result){  
            $("#par1").html(result)  
        },  
        error: function() {  
            alert("Something wrong");  
        }  
    });  
});
```

тоже самое

```
$("#par1").load("add.php","x=4&y=5")
```

```
$.ajax({  
  type: "GET",  
  url: "test.js",  
  dataType: "script"  
}); // Загружает и исполняет скрипт JavaScript
```

```
$.ajax({  
  type: "POST",  
  url: "some.php",  
  data: "name=John&location=Boston",  
  success: function(msg){  
    alert( "Data Saved: " + msg );  
  }  
}); //Загружает некоторые данные на сервер и оповещает пользователя по окончании операции.
```

```
bodyContent = $.ajax({  
    url: "script.php",  
    global: false,  
    type: "POST",  
    data: ({id : this.getAttribute('id')}),  
    dataType: "html",  
    success: function(msg){  
        alert(msg);  
    }  
})
```

).responseText; // Посылает идентификатор в качестве данных на сервер, сохраняет данные и оповещает пользователя по окончании операции.

jQuery.ajax

Это самый основной метод, а все последующие методы лишь обертки для метода jQuery.ajax. У данного метода лишь один входной параметр — объект включающий в себя все настройки (выделены параметры которые стоит запомнить):

- `async` — асинхронность запроса, по умолчанию `true`
- `cache` — вкл/выкл кэширование данных браузером, по умолчанию `true`
- `contentType` — по умолчанию «application/x-www-form-urlencoded»
- **`data`** — передаваемые данные — строка или объект
- `dataFilter` — фильтр для входных данных
- **`dataType`** — тип данных возвращаемых в callback функцию (`xml`, `html`, `script`, `json`, `text`, `_default`)
- `global` — триггер — отвечает за использование глобальных AJAX Event'ов, по умолчанию `true`
- `ifModified` — триггер — проверяет были ли изменения в ответе сервера, дабы не слать еще запрос, по умолчанию `false`

- `jsonp` — переустановить имя `callback` функции для работы с JSONP (по умолчанию генерируется на лету)
- `processData` — по умолчанию отправляемые данные заворачиваются в объект, и отправляются как «`application/x-www-form-urlencoded`», если надо иначе — отключаем
- `scriptCharset` — кодировочка — актуально для JSONP и загрузки JavaScript'ов
- `timeout` — время таймаут в миллисекундах
- **`type`** — GET либо POST
- **`url`** — url запрашиваемой страницы

Локальные **AJAX Event'ы**:

- `beforeSend` — срабатывает перед отправкой запроса
- `error` — если произошла ошибка
- `success` — если ошибок не возникло
- `complete` — срабатывает по окончании запроса

jQuery.getJSON

Загружает данные в формате JSON (удобней и быстрее нежели XML). Может принимать следующие параметры:

1. url запрашиваемой страницы
2. передаваемые данные (необязательный параметр)
3. callback функция, которой будет скормлен результат (необязательный параметр)

```
$(document).ready(function(){ // по завершению загрузки страницы
    $('#example-4').click(function(){ // вешаем на клик по элементу с id = example-4
        $.getJSON('ajax/example.json', {}, function(json){ // загрузку JSON данных из файла example.json
            $('#example-4').html("");
            // заполняем DOM элемент данными из JSON объекта
            $('#example-4').append('To: ' + json.note.to + '<br/>')
                .append('From: ' + json.note.from + '<br/>')
                .append('<b>' + json.note.heading + '</b><br/>')
                .append(json.note.body + '<br/>');
        });
    });
});
```

Файл example.json:

```
{  
  note:{  
    to:'Tove',  
    from:'Jani',  
    heading:'Reminder',  
    body:'Don\'t forget me this weekend!'  
  }  
}
```

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.3/jquery.min.js"></script>
```

```
<script>
```

```
function funcBefore() {
```

```
}
```

```
function funcSuccess(data) {
```

```
$("#information").text(data);
```

```
}
```

```
$(document).ready (function(){
```

```
$('#done').bind("click", function () {
```

```
var admin="ADmin";
```

```
$.ajax({
```

```
url:"check.php",
```

```
type: "POST",

data:({name: $('#name').val()}),

dataType:"html",

beforeSend: function ({

    $("#information").text("Waiting for Data...");

}),

success: funcSuccess

});

});

});

</script>
```

```
<body>
```

```
<input type="text" id="name" placeholder="name">
```

```
<input type="button" id="done" value="ok">
```

```
<p id="load" style="cursor:pointer">Loading Data</p>
```

```
<div id="information"></div>
```

```
</body>
```